SMART HAS

Consolidação: PL/SQL Avançado e DevOps

Packages · Triggers · Dynamic SQL · CI/CD (GitHub Actions)

Documentação da Atividade

FIAP — Consolidação do Projeto Smart HAS — 03/11/2026

Integrante

Nome completo	RM	Turma
Leonardo Marsiarelli	555366	3SIOA

Entrega individual.

1. Contextualização

O Smart HAS chegou à sua maturidade: app mobile em React Native, backend em Spring Boot com autenticação JWT e a camada AI Logistics Extension, dashboard administrativo em Angular, e uma camada de persistência e inteligência Oracle (package PL/SQL, triggers e Dynamic SQL) integrada ao backend via JDBC.

Nesta última etapa, o objetivo foi profissionalizar a entrega final: consolidar os objetos PL/SQL em um package organizado, adicionar automação via triggers, demonstrar Dynamic SQL para consultas flexíveis, e construir um pipeline de CI/CD que builda, testa e publica o backend automaticamente — com a integração Oracle validada a cada execução, não apenas revisada manualmente.

2. Parte 1 — Consolidação em Oracle PL/SQL

2.1 Package SMARTHAS_PKG

Todas as functions e procedures das fases anteriores foram agrupadas em um único package PL/SQL, garantindo organização (um ponto de entrada único, smarthas_pkg.xxx) e reuso (membros do package chamam uns aos outros sem repetir lógica — ex.: fn_calc_risk_score reutiliza fn_classify_bp internamente).

Membro	Tipo	Propósito
FN_CLASSIFY_BP	Function	Classifica a pressão arterial (reutilizada pelas demais)
FN_CALC_RISK_SCORE	Function	Indicador de risco (0-100) da AI Logistics Extension
FN_GET_USER_SUMMARY	Function	Resumo formatado do paciente, via CURSOR
PRC_REGISTER_CRITICAL_ALERT	Procedure	Registra alerta a partir de leitura crítica (aciona TRG_SENSOR_ALERT_AUDIT)
PRC_USER_DELIVERY_REPORT	Procedure	Relatório de entregas por paciente (CURSOR + LOOP + REF CURSOR)
PRC_SEARCH_READINGS	Procedure	Busca flexível de leituras — implementada com Dynamic SQL

2.2 Trigger de auditoria/automação

Duas triggers automatizam o registro de auditoria sem exigir nenhuma chamada explícita da aplicação — o rastro é gerado pelo próprio banco no momento em que o dado muda:

Trigger	Evento	Ação automática
TRG_SENSOR_ALERT_AUDIT	AFTER INSERT ON sensor_alert	Grava em AUDIT_LOG todo alerta criado (exemplo "registrar logs de alertas")

TRG_DELIVERY_STATUS_AUDIT	AFTER UPDATE OF status ON medication_delivery_request	Grava em AUDIT_LOG o antes/depois de cada mudança de status (exemplo "atualizar status automaticamente")

2.3 Dynamic SQL

PRC_SEARCH_READINGS resolve um problema que SQL estático não resolve bem: busca de leituras com filtros todos opcionais (paciente, classificação, data) e ordenação escolhida em tempo real. O WHERE é montado por concatenação (só entram os filtros informados) e o ORDER BY é resolvido contra uma whitelist fixa antes de ser concatenado — bind variable não parametriza nome de coluna, então não há como fazer isso sem SQL dinâmico.

```
v_order_col := CASE UPPER(p_order_by)
  WHEN 'RISK_DESC' THEN 'smarthas_pkg.fn_calc_risk_score(r.systolic, r.diastolic) DESC'
  ELSE 'r.measured_at DESC' END;
v_sql := 'SELECT ... FROM blood_pressure_reading r JOIN app_user u ON u.id=r.user_id WHERE 1=1';
IF p_user_id IS NOT NULL THEN v_sql := v_sql || ' AND r.user_id = :p_user_id'; END IF;
... OPEN p_cursor FOR v_sql USING <binds conforme os filtros ativos>;
```

Exposta via `GET /api/oracle/readings/search?userId=&classification=&since=&orderBy=`.

2.4 Documentação dos objetos e valor agregado

Cada function/procedure do package tem comentário de cabeçalho (propósito, parâmetros, retorno) e trata exceções com RAISE_APPLICATION_ERROR e códigos customizados (-20010 a -20040) — ver `database/03_package_smarthas.sql`. O valor agregado: regras de negócio centralizadas no banco, reutilizáveis por qualquer cliente (API, relatórios, BI), auditoria automática sem acoplamento com a aplicação, e consultas flexíveis sem precisar de múltiplos endpoints especializados.

3. Parte 2 — Automação e Entrega Contínua

Pipeline de CI/CD em GitHub Actions (`.github/workflows/ci-cd.yml`), disparado a cada push/PR para main, com três jobs encadeados:

Job	O que faz
build-test	Compila o backend, roda os testes automatizados (mvn test) e publica o JAR e o relatório de testes como artifacts
oracle-integration	Sobe um Oracle XE 21c real como serviço do job, implanta schema + seed + PACKAGE + TRIGGERS, e executa os exemplos de uso como teste de integração — falha o pipeline se aparecer qualquer erro ORA-/PLS-/SP2- não tratado
deploy	Builda a imagem Docker do backend (Dockerfile multi-stage) e publica no GitHub Container Registry, versionada por commit (ghcr.io/.../smart-has-backend:<sha>)

Logs e relatórios de execução: cada job grava um resumo em Step Summary, o relatório de testes (surefire) e o log completo da integração Oracle ficam disponíveis como artifacts do run, e o histórico completo de execuções fica na aba Actions do repositório.

3.1 Evidência de execução real

O pipeline foi executado de verdade no GitHub Actions (não apenas revisado como YAML): ver aba Actions do repositório para o histórico completo de runs, incluindo o job oracle-integration implantando o package/triggers em um Oracle real e rodando os exemplos de uso como teste de integração a cada push.

4. Entrega do Projeto

4.1 Código-fonte

Repositório GitHub: https://github.com/LeoMarsiarelli/Smart-HAS

Estrutura do monorepo: /mobile, /backend (inclui Dockerfile), /web-admin, /database (scripts PL/SQL), /.github/workflows (CI/CD), /docs.

4.2 Vídeo de demonstração

Link do vídeo (YouTube, não listado): [INSERIR LINK DO VÍDEO AQUI]

4.3 Credenciais de demonstração

Papel	Email	Senha
Administrador	admin@smarthas.com	admin123
Paciente	maria@smarthas.com	paciente123

5. Conclusão

O Smart HAS fecha esta etapa como um produto tecnicamente robusto: inteligência de negócio organizada em um package PL/SQL reutilizável, automação via triggers de auditoria, consultas flexíveis via Dynamic SQL, e um pipeline de CI/CD que builda, testa contra um Oracle real e publica a aplicação automaticamente a cada mudança — prático, auditável e pronto para evoluir rumo a um ambiente de mercado real.